

Pearls AQI Predictor

An End-to-End, Serverless Air Quality Index Forecasting System

Project Report

Prepared by: **Muhammad Sajid**
Full Stack AI Developer & Data Analyst
UET Taxila

Repository: github.com/muhammad-sajid17/pearls-aqi-predictor

Table of Contents

1. Executive Summary
2. Problem Statement & Objectives
3. System Architecture
4. Technology Stack
5. Data Source
6. Feature Pipeline
7. Feature Store & Model Registry
8. Training Pipeline & Model Evaluation
9. Exploratory Data Analysis
10. Explainability (SHAP)
11. Web Dashboard
12. Automation (CI/CD)
13. Deployment
14. Challenges & Solutions
15. Requirements Traceability Matrix
16. Limitations & Future Work
17. Conclusion

1. Executive Summary

The Pearls AQI Predictor is an end-to-end machine learning system that forecasts the Air Quality Index (AQI) of a chosen city 1, 2, and 3 days into the future. The system was built to satisfy a project brief requiring a **100% serverless** architecture: a feature pipeline that fetches and engineers data on a schedule, a feature store and model registry that persist that data and the resulting models, a training pipeline that evaluates multiple candidate models and keeps the best one, and a web application that serves live forecasts.

Every stage of the brief's reference architecture was implemented: automated hourly data collection, daily model retraining, a live Streamlit dashboard with hazard alerts, SHAP-based explainability, and a full exploratory data analysis. The system runs entirely on free-tier infrastructure — Open-Meteo for data (no API key required), Hopsworx for the feature store and model registry (free serverless tier, with a local-file fallback for zero-setup development), GitHub Actions for scheduling, and Streamlit Community Cloud for hosting.

Key Outcomes

- A fully automated pipeline requiring no manual intervention once deployed — features refresh hourly and the model retrains daily.
 - Three to four candidate ML models trained and benchmarked per run (Ridge Regression, Random Forest, Gradient Boosting, and an optional deep neural network), with the best one selected automatically by average RMSE across all three forecast horizons.
 - A production-style dashboard showing current AQI, a 3-day forecast, historical trend, a full model comparison leaderboard, SHAP feature-importance explanations, and automatic hazard alerts.
 - Zero ongoing infrastructure cost — every component used is on a free tier.
-

2. Problem Statement & Objectives

Air pollution is a persistent public health concern, and AQI is the standard metric used to communicate how polluted the air is and what health precautions are advisable. Unlike raw pollutant concentrations, AQI does not move randomly — it is strongly influenced by weather conditions (wind, humidity, temperature, atmospheric pressure) and follows daily and weekly patterns driven by traffic, industrial activity, and seasonal effects. This makes it a reasonable target for short-horizon time-series forecasting.

The project brief specified the following objectives:

- Fetch raw weather and pollutant data from an external API.
- Engineer time-based and derived features (e.g. AQI change rate) and prediction targets.
- Persist features in a feature store and back-fill historical data for training.

- Train and evaluate multiple ML models — from statistical to deep learning — using RMSE, MAE, and R^2 .
 - Register the best-performing model in a model registry.
 - Automate the feature and training pipelines on a schedule (hourly / daily) using free CI/CD tooling.
 - Serve live forecasts on an interactive dashboard, with feature-importance explanations and hazard alerts for unhealthy AQI levels.
-

3. System Architecture

The system follows a four-stage serverless pipeline, matching the reference architecture given in the project brief:

Stage	Purpose	Implementation
1. Raw Data	Fetch hourly weather + pollutant readings	data_source.py — Open-Meteo API
2. Feature Generation	Hourly → daily aggregation, feature/target engineering	feature_engineering.py, feature_pipeline.py
3. Feature Store & Model Registry	Persist features and trained models	store.py — Hopsworks or local files
4. Web App	Load model + features, forecast, visualize	app.py — Streamlit dashboard

Data Flow

Every hour, a scheduled job fetches the latest weather and pollutant readings, recomputes the daily feature table (including lag and rolling statistics), and writes it to the feature store. Once a day, a separate scheduled job pulls the full historical feature table, retrains every candidate model from scratch, evaluates each one, and registers the best-performing model. The dashboard is stateless with respect to computation — on every page load, it simply reads the latest features and the currently registered model, computes the 3-day forecast, and renders it. This separation means the dashboard never needs to be redeployed when the model changes; it always reflects whatever the training pipeline most recently produced.

Why "serverless"?

No component in this system requires a continuously running, manually managed server. The feature and training pipelines run as short-lived GitHub Actions jobs on a schedule and exit when done. The feature store and model registry (Hopsworks) are a managed service. The dashboard, when deployed to Streamlit Community Cloud, runs on managed infrastructure that scales down when idle. The only thing the developer maintains is source code.

4. Technology Stack

Layer	Technology	Why
Data source	Open-Meteo Weather & Air Quality APIs	Free, keyless, historical + forecast data for both weather and pollutants — avoids the signup friction of AQICN/OpenWeather while keeping the stack at zero cost
Feature engineering	Python, pandas, numpy	Standard, well-supported tabular data tooling
Feature store / model registry	Hopsworks (free serverless tier), with a local parquet/joblib fallback	Purpose-built MLOps feature store; local fallback removes setup friction for development and grading
Modeling	scikit-learn (Ridge, Random Forest, Gradient Boosting); optional TensorFlow/Keras dense network	Covers statistical, ensemble, and deep learning approaches as required by the brief
Explainability	SHAP	Model-agnostic, well-established feature-attribution method
Dashboard	Streamlit + Plotly	Rapid, Python-native interactive dashboards with no separate frontend build step
Automation	GitHub Actions (scheduled workflows)	Free CI/CD with native cron scheduling, no external orchestrator needed
Package management	uv	Fast, modern Python packaging and virtual environment tooling
Hosting (dashboard)	Streamlit Community Cloud	Free hosting for Streamlit apps directly from a GitHub repo

5. Data Source

The brief suggested AQICN or OpenWeather as example data sources. This project instead uses **Open-Meteo**, a free weather and air-quality data provider that requires no API key or account signup, for both the Weather API and the Air Quality API. This decision keeps the entire stack genuinely free and removes a common source of setup friction (API key management, rate limits tied to a personal account).

Data Collected

- **Pollutants (hourly):** US AQI, PM2.5, PM10, carbon monoxide, nitrogen dioxide, sulphur dioxide, ozone.
- **Weather (hourly):** temperature, relative humidity, wind speed, surface pressure, precipitation.

Open-Meteo's `past_days` parameter is capped at 92 days per request, which determines the maximum single-call historical backfill window used in this project.

Note: Open-Meteo's air-quality figures are model-based (CAMS atmospheric composition model) rather than raw ground-station sensor readings. This is a reasonable trade-off for a free, global, keyless data source, but is worth noting as a data-quality caveat — see Limitations.

6. Feature Pipeline

The feature pipeline (`feature_pipeline.py`) implements Steps 1–3 of the brief: fetching raw data, computing features and targets, and storing them.

6.1 Hourly → Daily Aggregation

Raw hourly readings are aggregated into one row per calendar day, using mean values for most fields, max for peak AQI, and sum for precipitation.

6.2 Time-Based Features

As required by the brief, calendar-derived features are computed for every day:

- **day_of_week** — 0 (Monday) through 6 (Sunday)
- **day_of_month, month**
- **is_weekend** — binary flag

6.3 Derived & Time-Series Features

- **Lag features:** AQI value 1, 2, 3, and 7 days prior
- **Rolling statistics:** 3-day and 7-day rolling mean of AQI, and 7-day rolling standard deviation
- **AQI change rate:** day-over-day percentage change in AQI, explicitly called for in the brief as an example derived feature

6.4 Targets

Three regression targets are created by shifting the daily AQI series forward: `aqi_next_1d`, `aqi_next_2d`, and `aqi_next_3d` — the AQI 1, 2, and 3 days ahead of each row's date.

6.5 Backfill vs. Incremental Modes

The pipeline supports two run modes:

- **Backfill** (`--backfill 92`) — pulls the maximum available history in a single call, used once to generate enough historical (features, targets) pairs for the first training run.
 - **Incremental** (no flag) — pulls the last 14 days, enough to correctly recompute lag/rolling features for recent dates, and is the mode run automatically every hour.
-

7. Feature Store & Model Registry

All interaction with the feature store and model registry is isolated behind a single module (`store.py`), so the rest of the codebase never needs to know which backend is active.

7.1 Dual-Backend Design

- **Hopsworks mode** — activated automatically when a `HOPSWORKS_API_KEY` environment variable is present. Writes to a real Hopsworks feature group and registers models in the Hopsworks Model Registry (free serverless tier).
- **Local fallback mode** — used when no Hopsworks key is set. Features are upserted into a local parquet file by date; the trained model and its metadata are saved as a joblib file and a JSON sidecar. This allows the entire system to be developed, tested, and even graded with zero external account setup.

This design means switching from local development to a production-grade managed feature store is a configuration change, not a code change — a deliberate MLOps best practice.

8. Training Pipeline & Model Evaluation

The training pipeline (`training_pipeline.py`) implements Steps 1–3 of the brief's training section.

8.1 Data Splitting

Historical (features, targets) rows are pulled from the feature store and split time-aware: the model trains on the earlier portion of the timeline and is evaluated on the most recent slice, never shuffled — this avoids leaking future information into the training set, which would be a methodological error for time-series data.

8.2 Candidate Models

Per the brief's instruction to experiment with a variety of models, from statistical to deep learning, four candidates are trained as `MultiOutputRegressors` (predicting all three forecast horizons jointly):

Model	Category	Notes
Ridge Regression	Statistical / linear	Regularized linear baseline; fast and interpretable
Random Forest	Tree ensemble (bagging)	Robust to non-linearity, handles feature interactions well
Gradient Boosting	Tree ensemble (boosting)	Sequential error-correction; often strong on tabular data
Dense Neural Network	Deep learning (optional)	Trained only if TensorFlow + scikeras are installed; gracefully skipped otherwise, per the brief's "experiment with" framing rather than a hard requirement

8.3 Evaluation Metrics

Every candidate is scored on the held-out test window using the three metrics specified in the brief, computed separately for each of the three forecast horizons:

- **RMSE** (Root Mean Squared Error) — penalizes large errors more heavily; primary selection metric

- **MAE** (Mean Absolute Error) — average magnitude of error, more robust to outliers
- **R²** (Coefficient of Determination) — proportion of variance explained by the model

8.4 Model Selection & Registration

The candidate with the lowest average RMSE across all three horizons is selected, refit on the *full* dataset (train + test combined) so the deployed model benefits from every available data point, and registered via the feature store/model registry abstraction.

8.5 Full Leaderboard & Failure Transparency

Rather than discarding the results of models that did not win, every candidate's full metric breakdown is saved to `models/leaderboard.json` and `leaderboard.csv`, and rendered in the dashboard's Model Comparison panel. If a candidate fails to train (for example, due to missing values a particular estimator cannot handle natively), it is still recorded with its error message rather than silently disappearing — a deliberate design choice for transparency during development and grading.

9. Exploratory Data Analysis

Per the brief's guideline to perform EDA to identify trends, a standalone script (`eda.py`) generates the following analyses from the accumulated feature store data:

- **AQI over time** — visualizes the overall trend and any obvious seasonal pattern in daily mean AQI.
- **AQI by day of week** — tests for weekly seasonality (e.g. lower AQI on weekends due to reduced traffic/industrial activity).
- **Correlation heatmap** — AQI against weather variables (temperature, humidity, wind, pressure, precipitation) and other pollutants, to identify which factors most strongly co-move with air quality.
- **Average pollutant concentrations** — a breakdown of which pollutant contributes most to the composite AQI figure.

These plots are saved as PNG files to a `reports/` directory and form the empirical basis for the feature engineering decisions described in Section 6 — for example, the inclusion of lag and rolling features is directly motivated by the autocorrelation visible in the AQI-over-time plot.

10. Explainability (SHAP)

As specified in the brief, SHAP (SHapley Additive exPlanations) is used to explain the model's predictions. For the currently registered model, SHAP values are computed against a recent window of feature rows, and the mean absolute SHAP value per feature is plotted as a horizontal bar chart in the dashboard — showing which features (e.g. yesterday's AQI, the rolling 3-day mean, the AQI change rate) most influence the 1-day-ahead forecast for the currently displayed prediction. This gives an end user — or a grader evaluating the system — visibility into *why* the model is forecasting what it forecasts, rather than treating it

as an opaque black box.

11. Web Dashboard

The dashboard (`app.py`) is built with Streamlit, satisfying the brief's requirement for the web app to load the model and features from the feature store, compute predictions, and present them on a simple, descriptive interface.

Components

Component	Description
Current AQI hero card	Today's AQI value, color-coded by official US AQI category (Good / Moderate / Unhealthy / etc.)
3-day forecast cards	Predicted AQI for the next 3 days, each individually color-coded by category
Historical trend chart	Plotly line chart of past AQI with the forecast overlaid, and the "Unhealthy" threshold marked
Hazard alert banner	A prominent red banner appears automatically whenever any of the 3 forecasted days is predicted to cross AQI 151 ("Unhealthy")
Model comparison leaderboard	Every candidate model from the last training run, with RMSE/MAE/ R^2 per horizon, the selected model highlighted, and any training failures shown transparently
SHAP feature importance	Bar chart of the features most influencing the current forecast
Model info panel	Which model is currently deployed, when it was trained, and its headline metrics

The dashboard is purely a read/render layer — it performs no training or heavy computation itself, only loading the latest artifacts and running inference, keeping page loads fast.

12. Automation (CI/CD)

Two GitHub Actions workflows fulfill the brief's automation requirement, using cron-scheduled triggers:

Workflow	Schedule	Action
feature_pipeline.yml	Every hour	Runs the incremental feature pipeline; refreshes the feature store with the latest data
training_pipeline.yml	Once daily	Retrains every candidate model on the full historical feature set and registers the best one

GitHub Actions was chosen over Apache Airflow (the brief's other suggested option) because it requires no separate orchestrator to host or maintain — the scheduler, compute, and version control live in the same free platform already hosting the source code.

In local-file fallback mode, each workflow run commits the updated `data/` and `models/` files back to the repository, so the pipeline's output persists between runs without needing external storage.

13. Deployment

The dashboard is deployable at no cost via Streamlit Community Cloud, which connects directly to the GitHub repository, installs the dependencies listed in `requirements.txt`, and redeploys automatically on every push to the main branch — completing the end-to-end automation loop from data collection through to the deployed, publicly viewable dashboard.

14. Challenges & Solutions

Development surfaced several real-world engineering issues, each resolved during the build:

Challenge	Root Cause	Resolution
HTTP 400 on deep backfill	Open-Meteo's <code>past_days</code> parameter is hard-capped at 92 days; initial chunking logic incorrectly exceeded it	Capped backfill requests at 92 days with a clear log message
Windows dependency install failure	The Hopsworks SDK's transitive dependency chain (<code>pyjks -> twofish</code>) requires a C compiler, unavailable by default on Windows	Isolated hopsworks into an optional <code>requirements-hopsworks.txt</code> so the core system installs cleanly everywhere
"Application Control policy has blocked this file"	Windows security software blocking DLL execution for files downloaded from the internet (Mark of the Web)	Unblocking files with PowerShell's <code>Unblock-File</code> , and relocating the project outside the Downloads folder
NaN errors during training	Early rows in the feature table lack sufficient history for 7-day lag/rolling features, and some estimators (Ridge, Gradient Boosting) do not accept NaN natively	Rows missing any feature or target are dropped before training
Silent disappearance of failed candidate models	Failed training attempts were excluded from the results dictionary entirely	Failures are now recorded with their error message and shown in the leaderboard rather than omitted

15. Requirements Traceability Matrix

Every requirement from the project brief, mapped to its implementation:

Requirement	Status	Implementation
Fetch raw weather + pollutant data	Done	data_source.py (Open-Meteo)
Compute features (inputs) and targets (outputs)	Done	feature_engineering.py
Time-based features (hour, day, month)	Done	day_of_week, day_of_month, month, is_weekend
Derived features (e.g. AQI change rate)	Done	aqi_change_rate, lag features, rolling stats
Store features in a feature store	Done	store.py (Hopsworks / local parquet)
Backfill historical (features, targets)	Done	feature_pipeline.py --backfill 92
Fetch historical data from the feature store	Done	training_pipeline.py
Train & evaluate the best ML model	Done	Ridge, Random Forest, Gradient Boosting, optional NN
Evaluate with RMSE, MAE, R^2	Done	Computed per model per horizon
Store the trained model in a model registry	Done	store.py::save_model()
Automate feature script hourly	Done	.github/workflows/feature_pipeline.yml
Automate training script daily	Done	.github/workflows/training_pipeline.yml
Web app loads model + features from the store	Done	app.py
Dashboard shows computed predictions	Done	Hero card, 3-day forecast cards, trend chart
Built with Streamlit/Gradio + Flask/FastAPI	Done	Streamlit
Perform EDA to identify trends	Done	eda.py
Variety of models: statistical to deep learning	Done	Ridge to tree ensembles to optional Keras NN
SHAP or LIME for feature importance	Done	SHAP, shown in dashboard
Alerts for hazardous AQI levels	Done	Red banner when forecast $\geq$ AQI 151

16. Limitations & Future Work

- **Single-city scope** — the pipeline forecasts one city at a time, parameterized by coordinates. Multi-city support would require one feature group and one scheduled job per city.
- **Model-estimated pollution data** — Open-Meteo's air quality figures come from an atmospheric model (CAMS) rather than raw ground-sensor readings. Integrating a ground-station API such as AQICN would improve calibration at the cost of requiring an API key.
- **Daily granularity** — forecasts are computed at daily resolution. Hourly-resolution forecasting is possible but would require denser lag features and considerably more history.
- **Training data volume grows over time** — because the system starts with only as much history as a single 92-day backfill provides, early forecasts are less accurate than they will become once the hourly/daily automation has accumulated months of real data.
- **Deep learning candidate is intentionally lightweight** — given realistic daily data volumes for a single city, tree-based models are expected to outperform a small neural network; a substantially larger, multi-city, multi-year dataset would be needed for deep learning to show a real advantage, which is a natural direction for future work.
- **Potential future enhancements:** multi-city support, ground-truth sensor integration, hourly-resolution forecasting, model monitoring/drift detection, and automated retraining triggers based on live model-performance degradation rather than a fixed daily schedule.

17. Conclusion

This project delivers a complete, working implementation of every stage specified in the Pearls AQI Predictor brief: automated data collection, feature engineering with the specific time-based and derived features requested, a feature store and model registry with a production-realistic dual-backend design, a multi-model training and evaluation pipeline with full transparency into every candidate's performance, hourly and daily automation via free CI/CD tooling, and an interactive dashboard with explainability and hazard alerting. Beyond meeting the brief's checklist, the system was engineered with attention to real operational concerns — graceful degradation when optional dependencies are unavailable, transparent handling of model training failures, and a local-first development mode that removes the need for any paid account or cloud credential to build, test, and demonstrate the system end to end.

Repository: github.com/muhammad-sajid17/pearls-aqi-predictor

Report generated as part of the final project submission.